PRÁCTICA 1

Mapas de memoria

Ampliación de Sistemas Operativos
2º curso del Grado en Ingeniería Informática

Universidad de León

Distributed under: Creative Commons Attribution-ShareAlike 4.0 International

1. Objetivos

El objetivo principal de esta práctica es entender de forma aplicada cómo está organizado el mapa de memoria de un proceso y como evoluciona durante la ejecución del mismo [1]. Para ello consultaremos la información contenida en `/proc/PID/maps`.

2. El archivo `/proc/PID/maps`

Linux ofrece un sistema de archivos especial: el sistema `/proc` [2, 3]. Este sistema de archivos no tiene soporte en ningún dispositivo. Su objetivo es dar al usuario datos sobre el estado del sistema en forma de archivos. En concreto la información de un proceso se encuentra en un directorio que tiene como nombre el Pid del proceso. E.g., la información del proceso con Pid 123 está en `/proc/123/`.

Además, para facilitar el acceso de un proceso a su propia información se crea un enlace a un directorio especial llamado `self`. E.g., cuando el proceso con Pid 123 accede a `/proc/self/` está accediendo a `/proc/123/`. En su directorio, cada proceso tiene archivos con mucha información. Nosotros nos centraremos en el fichero `maps`, que contiene el mapa de memoria el proceso.

Por ejemplo, si ejecutamos el comando "`cat /proc/self/maps`" obtendremos el mapa de memoria del proceso que está ejecutando `cat`, que será algo parecido a lo siguiente (puede variar en función de la versión del kernel):

```
$ cat /proc/self/maps
55f01b2db000-55f01b2e3000 r-xp 00000000 103:05 17825817  /bin/cat # Codigo ejecutable
55f01b4e2000-55f01b4e3000 r--p 00007000 103:05 17825817  /bin/cat # Constantes
55f01b4e3000-55f01b4e4000 rw-p 00008000 103:05 17825817  /bin/cat # Variables globales
55f01b6a2000-55f01b6c3000 rw-p 00000000 00:00 0          [heap]   # Reserva de memoria dinámica
7f90575b2000-7f90579c1000 r--p 00000000 103:05 263330    /usr/lib/locale/locale-archive
7f90579c1000-7f9057ba8000 r-xp 00000000 103:05 11542159  /lib/x86_64-linux-gnu/libc-2.27.so
7f9057ba8000-7f9057da8000 ---p 001e7000 103:05 11542159  /lib/x86_64-linux-gnu/libc-2.27.so
7f9057da8000-7f9057dac000 r--p 001e7000 103:05 11542159  /lib/x86_64-linux-gnu/libc-2.27.so
7f9057dac000-7f9057dae000 rw-p 001eb000 103:05 11542159  /lib/x86_64-linux-gnu/libc-2.27.so
7f9057dae000-7f9057db2000 rw-p 00000000 00:00 0
7f9057db2000-7f9057ddb000 r-xp 00000000 103:05 11540425  /lib/x86_64-linux-gnu/ld-2.27.so
7f9057f8d000-7f9057f8f000 rw-p 00000000 00:00 0
7f9057fb7000-7f9057fdb000 rw-p 00000000 00:00 0
7f9057fdb000-7f9057fdc000 r--p 00029000 103:05 11540425  /lib/x86_64-linux-gnu/ld-2.27.so
7f9057fdc000-7f9057fdd000 rw-p 0002a000 103:05 11540425  /lib/x86_64-linux-gnu/ld-2.27.so
7f9057fdd000-7f9057fde000 rw-p 00000000 00:00 0
7ffd8f965000-7ffd8f989000 rw-p 00000000 00:00 0          [stack]  # Variables locales
7ffd8f9ce000-7ffd8f9d1000 r--p 00000000 00:00 0          [vvar]
7ffd8f9d1000-7ffd8f9d2000 r-xp 00000000 00:00 0          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0  [vsyscall]
```

Cada línea describe una región del mapa de memoria del proceso. Y por cada región tenemos la siguiente información (columnas):

- Rango de direcciones de principio y fin (en hexadecimal) de la región.
- Protección de la región. Permisos: `r` (permiso de lectura), `w` (permiso de escritura) y `x` (permiso de ejecución). Y el tipo de región: `p` (privada) o `s` (compartida).
- Desplazamiento (offset) de la proyección en el archivo.
- En el caso de que sea una región con soporte, se especifica el dispositivo que contiene el archivo (mediante sus números Major y Minor), su i-nodo y su ruta absoluta. Si se trata de una región sin soporte, todos estos campos están a cero.

A partir de esta información podemos deducir a qué corresponde la mayor parte de las regiones del ejemplo anterior. La biblioteca `ld` se encarga de realizar todo lo referente al uso de bibliotecas dinámicas. `libc` es la biblioteca estándar de C usada por la mayoría de los procesos.

3. Ejercicios

La práctica consiste en desarrollar una serie de programas simples que realicen operaciones que afectan al mapa de memoria para poder analizar como evoluciona. Para ello, se incluirá en los puntos de interés de cada programa la impresión del mapa de memoria del proceso en ese instante. Por simplicidad, usaremos la función `system` para invocar el comando `cat` como se muestra en el siguiente ejemplo.

```
char cmd[25];
sprintf(cmd, "cat /proc/%d/maps", getpid());
system(cmd);
```

Además, se pedirá declarar cierto tipo de variables en diferentes partes de los ejercicios, para imprimir su dirección de memoria y poder compararla con los mapas de memoria que se imprimen en cada instante. Teniendo en cuenta esas direcciones de memoria, será necesario indicar en que región de la memoria se encuentran y si son instrucciones, variables globales (con o sin valor inicial), variables locales o constantes, del programa o las correspondientes librerías que se están usando en cada programa. Para simplificar la tarea, en cada ejercicio podréis encontrar una tabla que deberéis completar con estos datos.

A continuación se plantean los programas a implementar:

Programa 1

Implementa un programa que incluya: variables globales (con y sin valor inicial), variables locales, dentro del `main`, tanto de tipo escalar como vectores. Además el programa usará una variable externa al módulo. Concretamente la variable global `errno` (`#include <errno.h>`).

El programa debe imprimir su mapa de memoria y la dirección de todas las variables (incluida `errno`) y de la función `main`. En el caso de las variables también se pide su tamaño. Las direcciones de las variables y de la función `main` pueden imprimirse como sigue:

```
printf("La dirección de main es %p, la dirección de errno es %p y ocupa %ld Bytes", main,
    &errno, sizeof(errno));
```

Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante

Posible solución Programa 1

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>

const int constante = 33;
int a;
char b;
int c = 0;
char d = 'A';
int e[10];
char f[10];
int g[10] = {0,1,2,3,4,5,6,7,8,9};
char h[10] = {'a','b','c','d','e','f','g','h','i','j'};

int main() {
    char cmd[25];
    int i;
    static char j;
    int k = 33;
    static char l = 'B';
    int m[10];
    int n[10] = {0,1,2,3,4,5,6,7,8,9};

    sprintf(cmd, "cat /proc/%d/maps", getpid());
    system(cmd);

    printf("--\n");
    printf("Variable/función   Dirección              Tamaño\n");
    printf("            main   %p\n", main);
    printf("           errno   %p   %12ld\n", &errno, sizeof(errno));
    printf("       constante   %p   %12ld\n", &constante, sizeof(constante));
    printf("               a   %p   %12ld\n", &a, sizeof(a));
    printf("               b   %p   %12ld\n", &b, sizeof(b));
    printf("               c   %p   %12ld\n", &c, sizeof(c));
    printf("               d   %p   %12ld\n", &d, sizeof(d));
    printf("               e   %p   %12ld\n", &e, sizeof(e));
    printf("               f   %p   %12ld\n", &f, sizeof(f));
    printf("               g   %p   %12ld\n", &g, sizeof(g));
    printf("               h   %p   %12ld\n", &h, sizeof(h));
    printf("               i   %p   %12ld\n", &i, sizeof(i));
    printf("               j   %p   %12ld\n", &j, sizeof(j));
    printf("               k   %p   %12ld\n", &k, sizeof(k));
    printf("               l   %p   %12ld\n", &l, sizeof(l));
    printf("               m   %p   %12ld\n", &m, sizeof(m));
    printf("               n   %p   %12ld\n", &n, sizeof(n));

    return 0;
}
```

La salida que produce la ejecución del programa anterior es la siguiente. Además de mostrar la salida en la memoria hay que explicar en que región está cada una de las variables del programa. Esto lo haremos comparando su dirección (que hemos impreso debajo del mapa) con las direcciones de las diferentes regiones.

```
1)  55f0e2547000-55f0e2548000 r--p 00000000 103:05 19145567      /home/user/programa1
2)  55f0e2548000-55f0e2549000 r-xp 00001000 103:05 19145567      /home/user/programa1
3)  55f0e2549000-55f0e254a000 r--p 00002000 103:05 19145567      /home/user/programa1
4)  55f0e254a000-55f0e254b000 r--p 00002000 103:05 19145567      /home/user/programa1
5)  55f0e254b000-55f0e254c000 rw-p 00003000 103:05 19145567      /home/user/programa1
6)  7f34c1800000-7f34c1828000 r--p 00000000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
7)  7f34c1828000-7f34c19bd000 r-xp 00028000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
8)  7f34c19bd000-7f34c1a15000 r--p 001bd000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
9)  7f34c1a15000-7f34c1a16000 ---p 00215000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
10) 7f34c1a16000-7f34c1a1a000 r--p 00215000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
11) 7f34c1a1a000-7f34c1a1c000 rw-p 00219000 103:05 8128716       /usr/lib/x86_64-linux-gnu/libc.so.6
12) 7f34c1a1c000-7f34c1a29000 rw-p 00000000 00:00 0
13) 7f34c1a2f000-7f34c1a32000 rw-p 00000000 00:00 0
14) 7f34c1a61000-7f34c1a65000 rw-p 00000000 00:00 0
15) 7f34c1a65000-7f34c1a67000 r--p 00000000 103:05 8128633       /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
16) 7f34c1a67000-7f34c1a91000 r-xp 00002000 103:05 8128633       /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
17) 7f34c1a91000-7f34c1a9c000 r--p 0002c000 103:05 8128633       /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
18) 7f34c1a9d000-7f34c1a9f000 r--p 00037000 103:05 8128633       /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
19) 7f34c1a9f000-7f34c1aa1000 rw-p 00039000 103:05 8128633       /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
20) 7fffd2e24000-7fffd2e48000 rw-p 00000000 00:00 0              [stack]
21) 7fffd2fb1000-7fffd2fb5000 r--p 00000000 00:00 0              [vvar]
22) 7fffd2fb5000-7fffd2fb7000 r-xp 00000000 00:00 0              [vdso]
23) ffffffffff600000-ffffffffff601000 --xp 00000000 00:00        [vsyscall]
--
Variable/función   Dirección              Tamaño
            main   0x55f0e2548209
           errno   0x7f34c1a2f6c0              4
        constante  0x55f0e2549008              4
               a   0x55f0e254b0a0              4
               b   0x55f0e254b0a4              1
               c   0x55f0e254b0a8              4
               d   0x55f0e254b020              1
               e   0x55f0e254b0c0             40
               f   0x55f0e254b0e8             10
               g   0x55f0e254b040             40
               h   0x55f0e254b068             10
               i   0x7fffd2e42b08              4
               j   0x55f0e254b0f2              1
               k   0x7fffd2e42b0c              4
               l   0x55f0e254b072              1
               m   0x7fffd2e42b10             40
               n   0x7fffd2e42b40             40
```

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
0x55f0e2548209	**main**	-	2	X														
0x7f34c1a2f6c0	**errno**	4	13			X												
0x55f0e2549008	**constante**	4	3					X										
0x55f0e254b0a0	**a**	4	5		X													
0x55f0e254b0a4	**b**	1	5		X													
0x55f0e254b0a8	**c**	4	5		X													
0x55f0e254b020	**d**	1	5		X													
0x55f0e254b0c0	**e**	40	5		X													
0x55f0e254b0e8	**f**	10	5		X													
0x55f0e254b040	**g**	40	5		X													
0x55f0e254b068	**h**	10	5		X													
0x7fffd2e42b08	**i**	4	20				X											
0x55f0e254b0f2	**j**	1	5		X													
0x7fffd2e42b0c	**k**	4	20				X											
0x55f0e254b072	**l**	1	5		X													
0x7fffd2e42b10	**m**	40	20				X											
0x7fffd2e42b40	**n**	40	20				X											

Programa 2

Implementa un programa que incluya una función que tenga definida una variable local con un tamaño de 14.000 B (e.g. un vector de 14.000 caracteres). El programa debe imprimir y mostrar el mapa de memoria y las direcciones de main y de la función. Además, debe imprimir la dirección y el tamaño de la variable dentro de la función.

Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
main																		
función																		
vector																		

Programa 3

Implementa un programa que incluya una llamada a `malloc` que reserve 14.000 B y que luego los libere con `free`. El programa deberá imprimir el mapa de memoria, la **dirección**, el **contenido** y el **tamaño** de la variable, antes de la reserva, después de la reserva, y después de la liberación. Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
main																		
vector																		
Contenido del vector																		

Programa 4

Para comparar como afecta el uso de las librerías al mapa de memoria, se pide realizar 3 versiones de un programa que use la función coseno (`cos`) definida en la librería `libm`. Además de utilizar la función `cos` es necesario imprimir la dirección de memoria de la función y la dirección de memoria de la variable que almacena su contenido.

1. Versión compilada con librerías **estáticas**. El programa debe mostrar el mapa de memoria del proceso. En la compilación es necesario indicar que debe usarse la versión estática de la librería.

```
$ cc programa.c -static -lm -o programa
```

2. Versión compilada con librerías **dinámicas**. En este caso, para compilar es necesario indicar que debe usarse la versión dinámica de la librería (es la opción por defecto).

```
$ cc programa.c -lm -o programa
```

3. Versión que carga en tiempo de ejecución la librería matemática (usando `dlopen` [4]), obtiene la dirección de la función coseno (usando `dlsym`), la invoca y después cierra la librería (usando `dlclose`). El programa debe mostrar el mapa de memoria antes de `dlopen`, mientras está cargada la librería y después de `dlclose`.

   Para compilar, utilice el siguiente comando. Nótese que en el comando anterior no se especifica la biblioteca matemática, sino la librería que gestiona la carga explícita de librerías dinámicas.

```
$ cc programa.c -ldl -o programa
```

Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa				ld.so				libc.so				libm.so			
				Instrucción	Variable global	Variable local	Constante	Instrucción	Variable global	Variable local	Constante	Instrucción	Variable global	Variable local	Constante	Instrucción	Variable global	Variable local	Constante
Versión 1																			
Versión 2																			
Versión 3																			

Después de compilar los programas utiliza el comando `size` sobre los ejecutables y completa la siguiente tabla:

Versión	text	data	bss	dec	hex
1					
2					
3					

Programa 5

Implementa un programa que cree un hilo (utiliza la librería `pthread`), ver [5]. Crea una variable global y dentro de la función manejadora del hilo declara una variable local. El programa tiene que imprimir el mapa, antes de la creación (`pthread_create`), durante la existencia del mismo, y después de su finalización (`pthread_join`). Además en la función manejadora se debe imprimir la dirección y el tamaño de la variable local y la global. Para compilarlo:

```
$ cc programa.c -pthread -o programa
```

Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
Antes																		
Durante																		
Después																		

A continuación modifica el programa y crea un segundo hilo. Observa las diferencias del mapa y completa la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
Antes																		
Durante																		
Después																		

Programa 6

Implementa un programa que cree un proceso hijo (`fork`). El programa debe mostrar el mapa antes de crear el proceso hijo. Después, ambos procesos deben mostrar su mapa. Además, se debe crear una variable antes de la llamada a `fork` e imprimir su dirección y tamaño. Dentro de cada proceso crearemos otra variable e imprimiremos la dirección de memoria y el tamaño de todas ellas dentro de cada proceso.

La llamada a `fork` crea un nuevo proceso al duplicar el proceso que lo llama (proceso padre). El nuevo proceso se denomina proceso hijo. En el momento de fork() ambos espacios de memoria tienen el mismo contenido. El proceso hijo y el proceso padre se ejecutan en espacios de memoria separados y siguen caminos independientes. Si la llamada a fork() finaliza con éxito, el PID del proceso hijo se devuelve en el padre y se devolverá 0 en el hijo. En caso de fallo, no se crea ningún proceso hijo, y se devuelve -1 en el padre. A continuación se muestra un ejemplo de la llamada a `fork`.

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
  int pid = fork();
  if (pid==0)
    printf("Soy el hijo y mi pid es: %5d\n", getpid());
  else
    printf("Soy el padre, mi pid es: %5d y el pid de mi hijo es: %5d\n", getpid(), pid);

  return 0;
}
```

Con la salida del programa rellena la siguiente tabla:

Dirección	Variable / Función	sizeof()	Región	programa					ld.so					libc.so				
				Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante	Instrucción	Variable global CON valor inicial	Variable global SIN valor inicial	Variable local	Constante
Antes																		
Durante																		
Después																		

Programa 7

Implementa un programa que cree un proceso hijo (`fork`) y que este ejecute otro programa con `exec`. El hijo debe imprimir su mapa antes y después de invocar a `exec`. El programa que debe arrancar el hijo es `cat` sobre el archivo `maps` correspondiente.

Las funciones de la familia `exec` sustituyen la imagen del proceso actual por una nueva imagen. La nueva imagen se construye a partir de un fichero regular ejecutable. No hay valor de retorno en caso de que *exec* tenga éxito, porque la nueva imagen sustituye a la anterior. A continuación se muestran algunos ejemplos de como utilizar las funciones de la familia `exec`:

```
#include <unistd.h>

/* The following example executes the ls command, specifying the pathname of the
   executable (/bin/ls) and using arguments supplied directly to the command to produce
   single-column output. */
int ret;
ret = execl ("/bin/ls", "ls", "-l", (char *)0);

/* The following example searches for the location of the ls command among the directories
    specified by the PATH environment variable. */
int ret;
ret = execlp ("ls", "ls", "-l", (char *)0);
```

¿Puedes ver el mapa de memoria del proceso después de realizar la llamada a `exec`?

Programa 8

Implementa un programa que proyecte un archivo [1] utilizando las llamadas `mmap` y `munmap` [6]. El programa debe mostrar el mapa antes de la llamada a `mmap`, mientras el archivo esté proyectado, y después de la llamada a `munmap`. Prueba a cambiar algunos de los parámetros de `mmap` (compartido/privado, diferentes permisos, diferentes tamaños y desplazamientos, etc.) para después analizar como afectan al mapa.

Con la salida del programa rellena la siguiente tabla:

Instante	Número de regiones del mapa
Antes	
Durante	
Después	

Referencias

[1] Carretero. Prácticas de sistemas operativos. Mc Graw Hill.

[2] The /proc Filesystem, `https://www.kernel.org/doc/html/latest/filesystems/proc.html`. Online, last accessed March 2022.

[3] The Linux man-pages project, `https://www.kernel.org/doc/man-pages/`. Online, last accessed March 2022.

[4] dlopen(3) — Linux manual page, `https://man7.org/linux/man-pages/man3/dlopen.3.html`. Online, last accessed March 2022.

[5] pthread_create(3) — Linux manual page, `https://man7.org/linux/man-pages/man3/pthread_create.3.html`. Online, last accessed March 2022.

[6] mmap(2) — Linux manual page, `https://man7.org/linux/man-pages/man2/mmap.2.html`. Online, last accessed March 2022.